3. General Direct Illumination

Requirement Implement direct illumination equation

$$L_o(\mathbf{x}, \boldsymbol{\omega}_o) = \int_{\Omega} L_i(\mathbf{x}, \boldsymbol{\omega}_i) \, V(\mathbf{x}, \boldsymbol{\omega}_i) \, f(\mathbf{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \, |\cos\theta_i| \, d\boldsymbol{\omega}_i$$

We will now consider a few parameters in the Monte-Carlo class which encapsulates BRDFs. When the value of the parameter Monte-Carlo is 1 the the object's BRDF is **diffuse** and the diffuse reflection ρ_d is given by Monte-Carlo. If however Monte-Carlo is greater than 1 the object's BRDF is **specular** and the specular reflection ρ_s is given by Monte-Carlo. In the diffuse response we will the Monte-Carlo the corresponds the specular reflectance.

In this specialized setting, the BRDF in our direct illumination equation can be defined as:

$$f(\mathbf{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) = \begin{cases} \frac{\rho_d}{\pi}, & \text{if } \boldsymbol{\omega}_i = \mathbf{r} \\ \frac{\rho_s}{|\cos\theta_i|}, & \text{if } \boldsymbol{\omega}_i = \mathbf{r} \, \big| \, \mathrm{Cos}(\mathrm{acos}(\boldsymbol{\omega}_i \cdot \boldsymbol{\omega}_r)) \, \mathrm{Monte_Carlo} \\ 0, & \text{otherwise} \end{cases}$$

where the reflected view direction $\boldsymbol{\omega}_r = 2(\mathbf{n} \cdot \boldsymbol{\omega}_o)\mathbf{n} - \boldsymbol{\omega}_o$ and ρ_d is a constant from the shading parameters and ρ_s is the specular reflectance.

Note here that when checking the Monte-Carlo BRDF there are two cosine terms in your direct illumination equation: the Lambertian shading term above the normal, and the Phong cosine exponent reflection term. In the diffuse setting, the latter $\cos\alpha$ dependent $\cos\alpha$ is no longer present. Therefore, the Monte-Carlo BRDF is a function of the cosine of the angle between the reflected direction vector and the incoming direction. The second assignment deliverable *BRDF Importance Sampling*.

3.1 Environment Light Map

Requirement Load the cosineer and load **environment hyline**, which are implemented in the environment.py file. This time, we will implement an image-based lighting (IBL) scene. The environment map is a 2D RGB image corresponding to the (spherical) environment light that surrounds your scene, we will assume the environment is infinitely distant from every shading point.

You will implement the sphere, cos^2 function, which takes a key as input, and returns a key. $\cos^2$ function, which takes a key as input, and returns a key. Note that the key is a 3D vector, which is "interpreted" by the sky. The environment map we use are encoded in an **equal-rectangular** spherical parameterization according to:

$$x = 0.5 + \frac{1}{2\pi} \mathrm{atan2}(\mathbf{n}_x, \mathbf{n}_z) \, \mathrm{Flip_Horizontal} \, \big| \, \mathrm{Flip_Horizontal} \, \big| \, \mathrm{Tx}$$

$$y = 0.5 + \frac{1}{\pi} \mathrm{acos}(\mathbf{n}_y) \, \mathrm{Flip_Horizontal} \, \big| \, \mathrm{Flip_Horizontal} \, \big| \, \mathrm{Ty}$$

Note that both of the trigonometric functions above are available through the torch math module as far as torch and the sun.

4. Uniform Spherical Importance Sampling

The first Monte Carlo estimator you will implement is the simplest one, and will employ uniform spherical sampling. Recall that the factor that you can generate a uniformly-random ray direction $\boldsymbol{\omega} = (\boldsymbol{\omega}_x, \boldsymbol{\omega}_y, \boldsymbol{\omega}_z)$ on the sphere using two canonical random variables $\{\xi_1, \xi_2\} \sim \mathrm{computed}$ using $\mathrm{rand}() \sim \mathrm{computed}$ and the following transformation:

$$\boldsymbol{\omega}_z = 2\xi_1 - 1 \quad r = \sqrt{1 - \boldsymbol{\omega}_z^2} \quad \phi = 2\pi\xi_2$$

$$\boldsymbol{\omega}_x = r\cos\phi \quad \boldsymbol{\omega}_y = r\sin\phi$$

Here, the sampling PDF $p_{\mathrm{Uniform}}(\boldsymbol{\omega}) = \frac{1}{4\pi}$.

You can now proceed with computing your uniform spherical importance sampling MC estimator by tracing these sampled shadow rays $\mathbf{r} = \mathrm{from\ the\ shading\ point\ towards}\ \boldsymbol{\omega}_i \sim p_{\mathrm{Uniform}}(\boldsymbol{\omega})$ as:

$$L_o(\mathbf{x}, \boldsymbol{\omega}_o) = \frac{1}{N} \sum_{i=1}^{N} \frac{V(\mathbf{x}, \boldsymbol{\omega}_i) \, L_i(\mathbf{x}, \boldsymbol{\omega}_i) \, f(\mathbf{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \, |\cos\theta_i|}{p_{\mathrm{Uniform}}(\boldsymbol{\omega})}$$

Keep in mind that your integrator should only return an N = 1 sample estimate, as the progressive renderer (deliverable 5 will progressively render estimates with increasing N.

Uniform Spherical Importance Sampling @ 2D/2D: SSI SPP

5. BRDF Importance Sampling

The next MC estimator you will implement will employ BRDF importance sampling. Here, the sampling PDF you will implement will depend on whether you are in a diffuse or glossy Phong shading point, as:

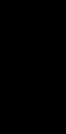
$$p_{\mathrm{BRDF}}(\boldsymbol{\omega}) = \begin{cases} \frac{1}{\pi} \, \big| \, \mathrm{Cos}(\mathrm{acos}(\mathbf{n} \cdot \boldsymbol{\omega})) & \text{if } \boldsymbol{\omega}_i = \mathbf{r} \\ \frac{\alpha+1}{2\pi} \, \big| \, \mathrm{Cos}(\mathrm{acos}(\mathbf{n} \cdot \boldsymbol{\omega})) \, \mathrm{Monte_Carlo} & \text{if } \boldsymbol{\omega}_i = \mathbf{r} \end{cases}$$

You can now sample $\boldsymbol{\omega}_i \sim p_{\mathrm{BRDF}}(\boldsymbol{\omega})$ in two ways. First drawing from a cosine-weighted hemisphere:

$$\boldsymbol{\omega}_z = \sqrt{\xi_1} \quad r = \sqrt{1 - \boldsymbol{\omega}_z^2} \quad \phi = 2\pi\xi_2$$

$$\boldsymbol{\omega}_x = r\cos\phi \quad \boldsymbol{\omega}_y = r\sin\phi$$

Keep in mind that your integrator should only return an N = 1 sample estimate, as the progressive renderer (deliverable 5 will progressively render estimates with increasing N.



Diffuse Surface Specular Surface

Sampled direction (yellow arrows) and BRDF lobes (blue outline) for diffuse and glossy Phong surfaces.

You can now similarly proceed with evaluating your BRDF importance sampling MC estimator by tracing these sampled shadow rays $\mathbf{r} = \mathrm{from\ your\ first\ point\ towards}\ \boldsymbol{\omega}_i \sim p_{\mathrm{BRDF}}(\boldsymbol{\omega}) \sim \mathrm{in}$:

$$L_o(\mathbf{x}, \boldsymbol{\omega}_o) = \frac{1}{N} \sum_{i=1}^{N} \frac{V(\mathbf{x}, \boldsymbol{\omega}_i) \, L_i(\mathbf{x}, \boldsymbol{\omega}_i) \, f(\mathbf{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \, |\cos\theta_i|}{p_{\mathrm{BRDF}}(\boldsymbol{\omega})}$$

keeping in mind that, depending on the type of surface, the PDF will "cancel out" differently, since for diffuse surfaces, $f(\mathbf{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \, |\cos\theta_i| = \frac{\rho_d}{\pi} \, |\cos\theta_i|$ will simplify to ρ_d and for glossy Phong surfaces it will simplify to $\rho_s \, \mathrm{Monte_Carlo}$. While you can implement these two estimators separately, you should combine them into a single estimator, simply multiplying the MC estimator expression above will yield the same result.

As before, your integrator should only output a single N = 1 sample estimate, as your progressive rendering tool will appropriately accumulate and average these individual estimates.

BRDF Importance Sampling @ 2D/2D: SSI SPP

Deliverable 2 (10 points)

- Implement the shading logic of both uniform spherical importance sampling and BRDF importance sampling in the diffuse/glossy in a single, cosine function
- The shading function will either employ the uniform or BRDF importance sampling based on the Monte-Carlo Phong.